VIT-AP UNIVERSITY, ANDHRA PRADESH

Lab Sheet 6: MongoDB Basic commands

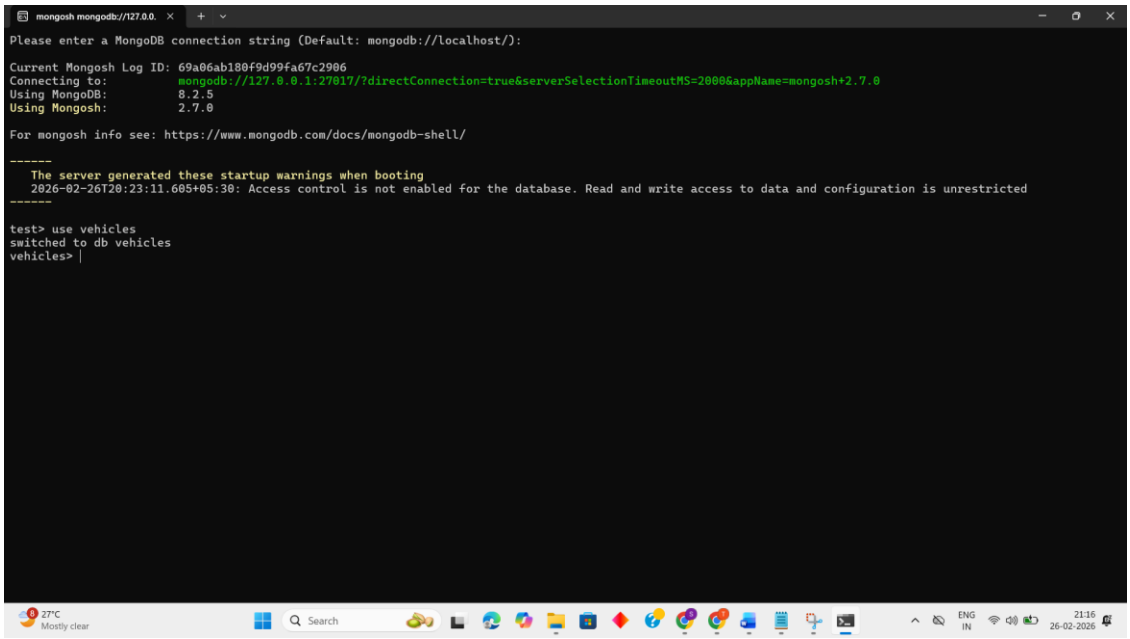
Branch/ Class: B.Tech
Faculty Name: Prof. S.Gopikrishnan
Student name: T. Sri Durga Priya

Date: 26/02/2026
School: SCOPE
Reg. no.: 23BCE9881

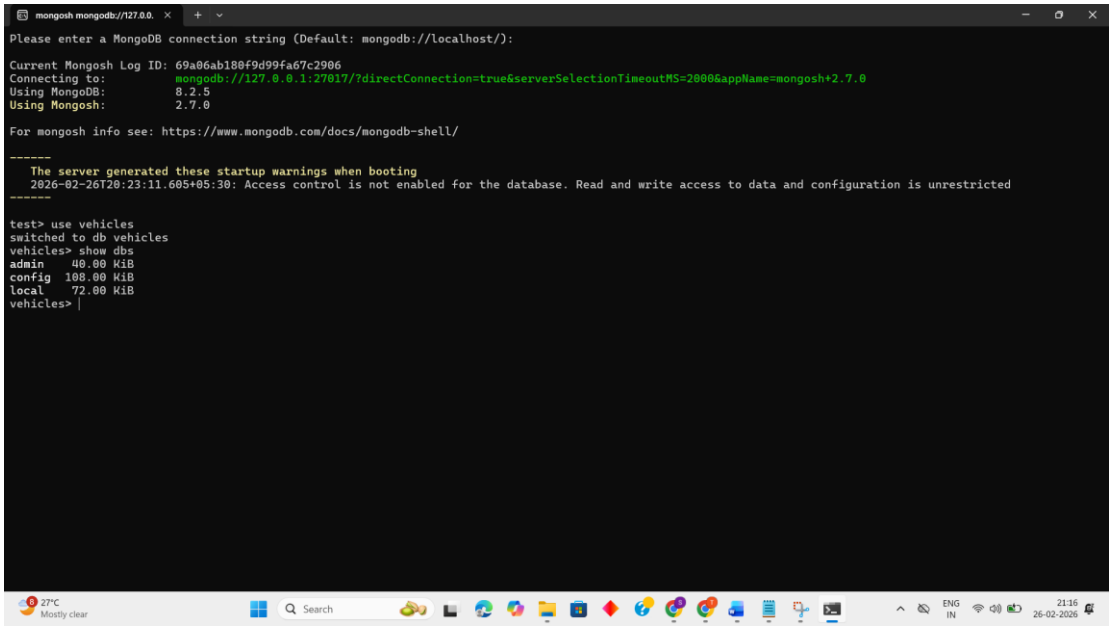
---

1. Use MongoDB to implement the following DB operations

   1. Create a database called 'vehicles' and *write* a MongoDB query to select database as "vehicles".



   2. Write a MongoDB query to display all the databases.



3. Create a collection called 'two_wheelers'. (use capping) and Create a collection called 'four_wheelers'.

```
mongosh mongodb://127.0.0.

Please enter a MongoDB connection string (Default: mongodb://localhost/):

Current Mongosh Log ID: 69a06ab180f9d99fa67c2906
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB:          8.2.5
Using Mongosh:          2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-02-26T20:23:11.605+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
------

test> use vehicles
switched to db vehicles
vehicles> show dbs
admin    40.00 KiB
config  108.00 KiB
local    72.00 KiB
vehicles> db.createCollection("two_wheelers", {
|   capped: true,
|   size: 500000,
|   max: 50
| })
{ ok: 1 }
vehicles> db.createCollection("four_wheelers")
{ ok: 1 }
vehicles>
```

4. Add 5 two-wheeler details to the collection named 'two_wheelers'. Each document consists of following fields as bike_name, model (gear or gearless), category (100cc, 125cc, 150cc, 200cc), colors_available (red, black, blue, sport red etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

```
mongosh mongodb://127.0.0.

vehicles> db.two_wheelers.insertMany([
| {
|   bike_name: "Meteor Glide 125",
|   model: "gear",
|   category: "125cc",
|   colors_available: ["crimson red", "jet black"],
|   manufacturer: "Skyline Motors",
|   performance: 7,
|   timestamp: new Date("2021-03-15"),
|   price: 92000
| },
| {
|   bike_name: "Urban Zip 100",
|   model: "gearless",
|   category: "100cc",
|   colors_available: ["pearl white", "aqua blue"],
|   manufacturer: "Velocity Auto",
|   performance: 6,
|   timestamp: new Date("2020-11-10"),
|   price: 68000
| },
| {
|   bike_name: "Ranger Pro 200",
|   model: "gear",
|   category: "200cc",
|   colors_available: ["forest green", "matte black"],
|   manufacturer: "TrailBlaze",
|   performance: 9,
|   timestamp: new Date("2022-06-18"),
|   price: 158000
| },
| {
|   bike_name: "Flash Rider 150",
|   model: "gear",
|   category: "150cc",
|   colors_available: ["sport red", "electric blue"],
|   manufacturer: "Nova Wheels",
|   performance: 8,
|   timestamp: new Date("2023-01-05"),
|   price: 120000
| },
```

```
|   bike_name: "Eco Spark 125",
|   model: "gearless",
|   category: "125cc",
|   colors_available: ["silver grey", "midnight blue"],
|   manufacturer: "GreenRide",
|   performance: 7,
|   timestamp: new Date("2022-09-22"),
|   price: 88000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69a06b5880f9d99fa67c2907'),
    '1': ObjectId('69a06b5880f9d99fa67c2908'),
    '2': ObjectId('69a06b5880f9d99fa67c2909'),
    '3': ObjectId('69a06b5880f9d99fa67c290a'),
    '4': ObjectId('69a06b5880f9d99fa67c290b')
  }
}
vehicles> |
```

5. Add 5 four-wheeler details to the collection named 'four_wheelers'. Each document consists of following fields as vehicle_name, model (commercial or own), category (car, lorry, bus, mini truck, heavy truck, containers), variants (vxi, zxi, petrol, diesel etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

```
vehicles> db.four_wheelers.insertMany([
| {
|   vehicle_name: "Orion X Sedan",
|   model: "own",
|   category: "car",
|   variants: ["vxi", "petrol", "automatic"],
|   manufacturer: "Orion Motors",
|   performance: 8,
|   timestamp: new Date("2023-04-12"),
|   price: 850000
| },
| {
|   vehicle_name: "Titan Hauler 400",
|   model: "commercial",
|   category: "heavy truck",
|   variants: ["diesel", "container"],
|   manufacturer: "Titan AutoCorp",
|   performance: 9,
|   timestamp: new Date("2021-07-19"),
|   price: 4200000
| },
| {
|   vehicle_name: "CityMove Mini",
|   model: "commercial",
|   category: "mini truck",
|   variants: ["diesel", "manual"],
|   manufacturer: "Metro Vehicles",
|   performance: 7,
|   timestamp: new Date("2020-02-25"),
|   price: 650000
| },
| {
|   vehicle_name: "AeroBus 320",
|   model: "commercial",
|   category: "bus",
|   variants: ["diesel", "school edition"],
|   manufacturer: "Aero Transport",
|   performance: 8,
|   timestamp: new Date("2022-10-30"),
|   price: 3100000
| },
```

```
|   vehicle_name: "CityMove Mini",
|   model: "commercial",
|   category: "mini truck",
|   variants: ["diesel", "manual"],
|   manufacturer: "Metro Vehicles",
|   performance: 7,
|   timestamp: new Date("2020-02-25"),
|   price: 650000
| },
| {
|   vehicle_name: "AeroBus 320",
|   model: "commercial",
|   category: "bus",
|   variants: ["diesel", "school edition"],
|   manufacturer: "Aero Transport",
|   performance: 8,
|   timestamp: new Date("2022-10-30"),
|   price: 3100000
| },
| {
|   vehicle_name: "Zenith LX",
|   model: "own",
|   category: "car",
|   variants: ["zxi", "diesel"],
|   manufacturer: "Zenith Automotive",
|   performance: 6,
|   timestamp: new Date("2019-12-14"),
|   price: 780000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69a06b8a80f9d99fa67c290c'),
    '1': ObjectId('69a06b8a80f9d99fa67c290d'),
    '2': ObjectId('69a06b8a80f9d99fa67c290e'),
    '3': ObjectId('69a06b8a80f9d99fa67c290f'),
    '4': ObjectId('69a06b8a80f9d99fa67c2910')
  }
}
vehicles> |
```

6. Write a MongoDB query to display all documents available in two_wheelers and four_wheelers.

```
mongosh mongodb://127.0.0.
  }
}
vehicles> db.two_wheelers.find()
[
  {
    _id: ObjectId('69a06b5880f9d99fa67c2907'),
    bike_name: 'Meteor Glide 125',
    model: 'gear',
    category: '125cc',
    colors_available: [ 'crimson red', 'jet black' ],
    manufacturer: 'Skyline Motors',
    performance: 7,
    timestamp: ISODate('2021-03-15T00:00:00.000Z'),
    price: 92000
  },
  {
    _id: ObjectId('69a06b5880f9d99fa67c2908'),
    bike_name: 'Urban Zip 100',
    model: 'gearless',
    category: '100cc',
    colors_available: [ 'pearl white', 'aqua blue' ],
    manufacturer: 'Velocity Auto',
    performance: 6,
    timestamp: ISODate('2020-11-10T00:00:00.000Z'),
    price: 68000
  },
  {
    _id: ObjectId('69a06b5880f9d99fa67c2909'),
    bike_name: 'Ranger Pro 200',
    model: 'gear',
    category: '200cc',
    colors_available: [ 'forest green', 'matte black' ],
    manufacturer: 'TrailBlaze',
    performance: 9,
    timestamp: ISODate('2022-06-18T00:00:00.000Z'),
    price: 158000
  },
  {
    _id: ObjectId('69a06b5880f9d99fa67c290a'),
    bike_name: 'Flash Rider 150',
    model: 'gear',
```

```
mongosh mongodb://127.0.0.
    model: 'gear',
    category: '150cc',
    colors_available: [ 'sport red', 'electric blue' ],
    manufacturer: 'Nova Wheels',
    performance: 8,
    timestamp: ISODate('2023-01-05T00:00:00.000Z'),
    price: 120000
  },
  {
    _id: ObjectId('69a06b5880f9d99fa67c290b'),
    bike_name: 'Eco Spark 125',
    model: 'gearless',
    category: '125cc',
    colors_available: [ 'silver grey', 'midnight blue' ],
    manufacturer: 'GreenRide',
    performance: 7,
    timestamp: ISODate('2022-09-22T00:00:00.000Z'),
    price: 88000
  }
]
vehicles> db.four_wheelers.find()
[
  {
    _id: ObjectId('69a06b8a80f9d99fa67c290c'),
    vehicle_name: 'Orion X Sedan',
    model: 'own',
    category: 'car',
    variants: [ 'vxi', 'petrol', 'automatic' ],
    manufacturer: 'Orion Motors',
    performance: 8,
    timestamp: ISODate('2023-04-12T00:00:00.000Z'),
    price: 850000
  },
  {
    _id: ObjectId('69a06b8a80f9d99fa67c290d'),
    vehicle_name: 'Titan Hauler 400',
    model: 'commercial',
    category: 'heavy truck',
    variants: [ 'diesel', 'container' ],
    manufacturer: 'Titan AutoCorp',
    performance: 9,
```

```
mongosh mongodb://127.0.0.
    variants: [ 'diesel', 'container' ],
    manufacturer: 'Titan AutoCorp',
    performance: 9,
    timestamp: ISODate('2021-07-19T00:00:00.000Z'),
    price: 4200000
  },
  {
    _id: ObjectId('69a06b8a80f9d99fa67c290e'),
    vehicle_name: 'CityMove Mini',
    model: 'commercial',
    category: 'mini truck',
    variants: [ 'diesel', 'manual' ],
    manufacturer: 'Metro Vehicles',
    performance: 7,
    timestamp: ISODate('2020-02-25T00:00:00.000Z'),
    price: 650000
  },
  {
    _id: ObjectId('69a06b8a80f9d99fa67c290f'),
    vehicle_name: 'AeroBus 320',
    model: 'commercial',
    category: 'bus',
    variants: [ 'diesel', 'school edition' ],
    manufacturer: 'Aero Transport',
    performance: 8,
    timestamp: ISODate('2022-10-30T00:00:00.000Z'),
    price: 3100000
  },
  {
    _id: ObjectId('69a06b8a80f9d99fa67c2910'),
    vehicle_name: 'Zenith LX',
    model: 'own',
    category: 'car',
    variants: [ 'zxi', 'diesel' ],
    manufacturer: 'Zenith Automotive',
    performance: 6,
    timestamp: ISODate('2019-12-14T00:00:00.000Z'),
    price: 780000
  }
]
vehicles>
```

7. Write a MongoDB query to display only vehicle name and price in all the collection of the database

```
mongosh mongodb://127.0.0.
  },
  {
    _id: ObjectId('69a06b8a80f9d99fa67c290f'),
    vehicle_name: 'AeroBus 320',
    model: 'commercial',
    category: 'bus',
    variants: [ 'diesel', 'school edition' ],
    manufacturer: 'Aero Transport',
    performance: 8,
    timestamp: ISODate('2022-10-30T00:00:00.000Z'),
    price: 3100000
  },
  {
    _id: ObjectId('69a06b8a80f9d99fa67c2910'),
    vehicle_name: 'Zenith LX',
    model: 'own',
    category: 'car',
    variants: [ 'zxi', 'diesel' ],
    manufacturer: 'Zenith Automotive',
    performance: 6,
    timestamp: ISODate('2019-12-14T00:00:00.000Z'),
    price: 780000
  }
]
vehicles> db.two_wheelers.find({}, { bike_name: 1, price: 1, _id: 0 })
[
  { bike_name: 'Meteor Glide 125', price: 92000 },
  { bike_name: 'Urban Zip 100', price: 68000 },
  { bike_name: 'Ranger Pro 200', price: 158000 },
  { bike_name: 'Flash Rider 150', price: 120000 },
  { bike_name: 'Eco Spark 125', price: 88000 }
]
vehicles> db.four_wheelers.find({}, { vehicle_name: 1, price: 1, _id: 0 })
[
  { vehicle_name: 'Orion X Sedan', price: 850000 },
  { vehicle_name: 'Titan Hauler 400', price: 4200000 },
  { vehicle_name: 'CityMove Mini', price: 650000 },
  { vehicle_name: 'AeroBus 320', price: 3100000 },
  { vehicle_name: 'Zenith LX', price: 780000 }
]
vehicles>
```

8. Write a MongoDB query to display two_wheelers from a particular company

```
mongosh mongodb://127.0.0.
    vehicle_name: 'Zenith LX',
    model: 'own',
    category: 'car',
    variants: [ 'zxi', 'diesel' ],
    manufacturer: 'Zenith Automotive',
    performance: 6,
    timestamp: ISODate('2019-12-14T00:00:00.000Z'),
    price: 780000
  }
]
vehicles> db.two_wheelers.find({}, { bike_name: 1, price: 1, _id: 0 })
[
  { bike_name: 'Meteor Glide 125', price: 92000 },
  { bike_name: 'Urban Zip 100', price: 68000 },
  { bike_name: 'Ranger Pro 200', price: 158000 },
  { bike_name: 'Flash Rider 150', price: 120000 },
  { bike_name: 'Eco Spark 125', price: 88000 }
]
vehicles> db.four_wheelers.find({}, { vehicle_name: 1, price: 1, _id: 0 })
[
  { vehicle_name: 'Orion X Sedan', price: 850000 },
  { vehicle_name: 'Titan Hauler 400', price: 4200000 },
  { vehicle_name: 'CityMove Mini', price: 650000 },
  { vehicle_name: 'AeroBus 320', price: 3100000 },
  { vehicle_name: 'Zenith LX', price: 780000 }
]
vehicles> db.two_wheelers.find({ manufacturer: "Skyline Motors" })
[
  {
    _id: ObjectId('69a06b5880f9d99fa67c2907'),
    bike_name: 'Meteor Glide 125',
    model: 'gear',
    category: '125cc',
    colors_available: [ 'crimson red', 'jet black' ],
    manufacturer: 'Skyline Motors',
    performance: 7,
    timestamp: ISODate('2021-03-15T00:00:00.000Z'),
    price: 92000
  }
]
vehicles> |
```

9. Write a MongoDB query to display four_wheelers available in diesel variants

```
mongosh mongodb://127.0.0.
vehicles> db.four_wheelers.find({ variants: "diesel" })
[
  {
    _id: ObjectId('69a06b8a80f9d99fa67c290d'),
    vehicle_name: 'Titan Hauler 400',
    model: 'commercial',
    category: 'heavy truck',
    variants: [ 'diesel', 'container' ],
    manufacturer: 'Titan AutoCorp',
    performance: 9,
    timestamp: ISODate('2021-07-19T00:00:00.000Z'),
    price: 4200000
  },
  {
    _id: ObjectId('69a06b8a80f9d99fa67c290e'),
    vehicle_name: 'CityMove Mini',
    model: 'commercial',
    category: 'mini truck',
    variants: [ 'diesel', 'manual' ],
    manufacturer: 'Metro Vehicles',
    performance: 7,
    timestamp: ISODate('2020-02-25T00:00:00.000Z'),
    price: 650000
  },
  {
    _id: ObjectId('69a06b8a80f9d99fa67c290f'),
    vehicle_name: 'AeroBus 320',
    model: 'commercial',
    category: 'bus',
    variants: [ 'diesel', 'school edition' ],
    manufacturer: 'Aero Transport',
    performance: 8,
    timestamp: ISODate('2022-10-30T00:00:00.000Z'),
    price: 3100000
  },
  {
    _id: ObjectId('69a06b8a80f9d99fa67c2910'),
    vehicle_name: 'Zenith LX',
    model: 'own',
    category: 'car',
    variants: [ 'zxi', 'diesel' ],
```

```
  {
    _id: ObjectId('69a06b8a80f9d99fa67c2910'),
    vehicle_name: 'Zenith LX',
    model: 'own',
    category: 'car',
    variants: [ 'zxi', 'diesel' ],
    manufacturer: 'Zenith Automotive',
    performance: 6,
    timestamp: ISODate('2019-12-14T00:00:00.000Z'),
    price: 780000
  }
]
vehicles> |
```

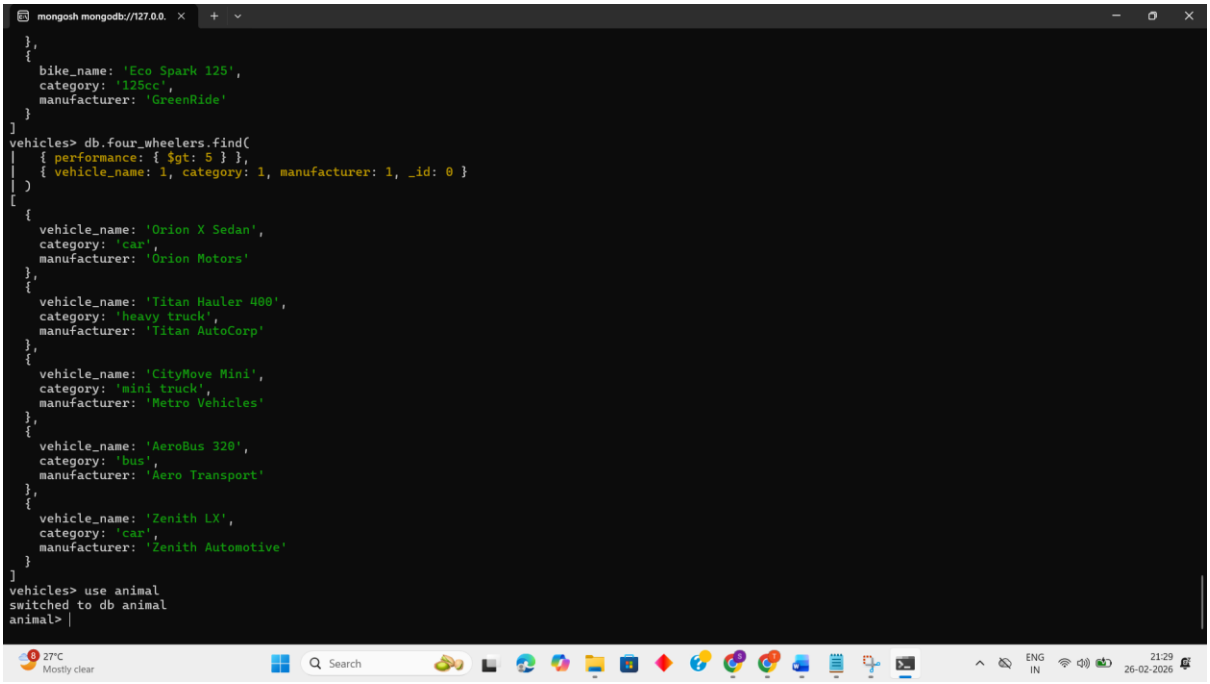
10. Write a MongoDB query to display vehicles name, category and manufacturer details whose rating is more than 5.

```
mongosh mongodb://127.0.0.
    model: 'own',
    category: 'car',
    variants: [ 'zxi', 'diesel' ],
    manufacturer: 'Zenith Automotive',
    performance: 6,
    timestamp: ISODate('2019-12-14T00:00:00.000Z'),
    price: 780000
  }
]
vehicles> db.two_wheelers.find(
|   { performance: { $gt: 5 } },
|   { bike_name: 1, category: 1, manufacturer: 1, _id: 0 }
| )
[
  {
    bike_name: 'Meteor Glide 125',
    category: '125cc',
    manufacturer: 'Skyline Motors'
  },
  {
    bike_name: 'Urban Zip 100',
    category: '100cc',
    manufacturer: 'Velocity Auto'
  },
  {
    bike_name: 'Ranger Pro 200',
    category: '200cc',
    manufacturer: 'TrailBlaze'
  },
  {
    bike_name: 'Flash Rider 150',
    category: '150cc',
    manufacturer: 'Nova Wheels'
  },
  {
    bike_name: 'Eco Spark 125',
    category: '125cc',
    manufacturer: 'GreenRide'
  }
]
vehicles> |
```

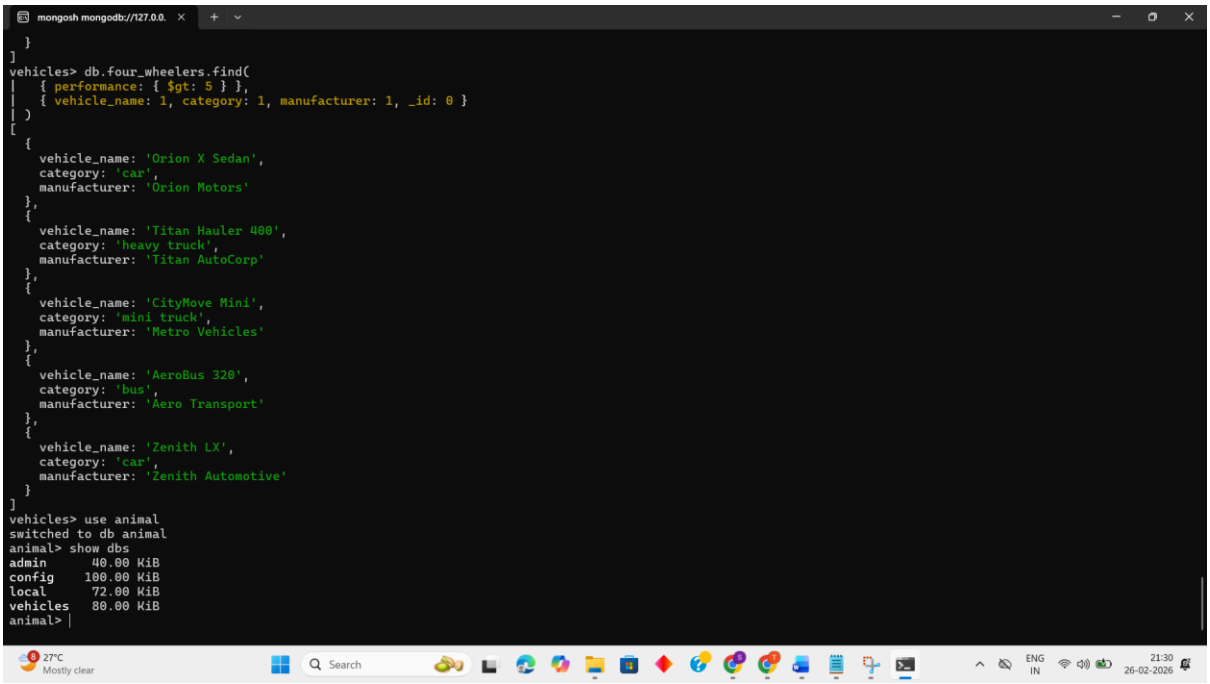
```
mongosh mongodb://127.0.0.
    category: '150cc',
    manufacturer: 'Nova Wheels'
  },
  {
    bike_name: 'Eco Spark 125',
    category: '125cc',
    manufacturer: 'GreenRide'
  }
]
vehicles> db.four_wheelers.find(
|   { performance: { $gt: 5 } },
|   { vehicle_name: 1, category: 1, manufacturer: 1, _id: 0 }
| )
[
  {
    vehicle_name: 'Orion X Sedan',
    category: 'car',
    manufacturer: 'Orion Motors'
  },
  {
    vehicle_name: 'Titan Hauler 400',
    category: 'heavy truck',
    manufacturer: 'Titan AutoCorp'
  },
  {
    vehicle_name: 'CityMove Mini',
    category: 'mini truck',
    manufacturer: 'Metro Vehicles'
  },
  {
    vehicle_name: 'AeroBus 320',
    category: 'bus',
    manufacturer: 'Aero Transport'
  },
  {
    vehicle_name: 'Zenith LX',
    category: 'car',
    manufacturer: 'Zenith Automotive'
  }
]
vehicles> |
```

2. Use MongoDB to implement the following DB operations for a Zoo

   1. Create a database called 'animal' and *write* a MongoDB query to select database as 'animal'.



   2. Write a MongoDB query to display all the databases.



3. Create a collection called 'wild_animals'.(use capping) and Create a collection called 'domestic_animals'.

```
mongosh mongodb://127.0.0.
    vehicle_name: 'Orion X Sedan',
    category: 'car',
    manufacturer: 'Orion Motors'
  },
  {
    vehicle_name: 'Titan Hauler 400',
    category: 'heavy truck',
    manufacturer: 'Titan AutoCorp'
  },
  {
    vehicle_name: 'CityMove Mini',
    category: 'mini truck',
    manufacturer: 'Metro Vehicles'
  },
  {
    vehicle_name: 'AeroBus 320',
    category: 'bus',
    manufacturer: 'Aero Transport'
  },
  {
    vehicle_name: 'Zenith LX',
    category: 'car',
    manufacturer: 'Zenith Automotive'
  }
]
vehicles> use animal
switched to db animal
animal> show dbs
admin      40.00 KiB
config    100.00 KiB
local      72.00 KiB
vehicles   80.00 KiB
animal> db.createCollection("wild_animals", {
|   capped: true,
|   size: 400000,
|   max: 50
| })
{ ok: 1 }
animal> db.createCollection("domestic_animals")
{ ok: 1 }
animal>
```

4. Add 5 wild_animal details to the collection named 'wild_animals'. Each document consists of following fields as animal_name, nature (harm or harmless), favorite_foods (meat, rabbits, deer etc) as array, care_taker_name, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
mongosh mongodb://127.0.0.
animal> db.wild_animals.insertMany([
| {
|   animal_name: "White Bengal Tiger",
|   nature: "harm",
|   favorite_foods: ["meat", "deer", "rabbit"],
|   care_taker_name: "Raghav Sharma",
|   life_span: 15,
|   timestamp: new Date("2019-06-12"),
|   expenses: 50000
| },
| {
|   animal_name: "African Lion",
|   nature: "harm",
|   favorite_foods: ["meat", "zebra"],
|   care_taker_name: "Aman Verma",
|   life_span: 14,
|   timestamp: new Date("2020-03-22"),
|   expenses: 47000
| },
| {
|   animal_name: "Spotted Deer",
|   nature: "harmless",
|   favorite_foods: ["grass", "leaves"],
|   care_taker_name: "Suman Patel",
|   life_span: 10,
|   timestamp: new Date("2021-09-18"),
|   expenses: 15000
| },
| {
|   animal_name: "Grey Wolf",
|   nature: "harm",
|   favorite_foods: ["meat", "rabbit"],
|   care_taker_name: "Raghav Sharma",
|   life_span: 12,
|   timestamp: new Date("2018-11-05"),
|   expenses: 30000
| },
| {
|   animal_name: "Peacock",
|   nature: "harmless",
|   favorite_foods: ["grains", "insects"],
```

```
mongosh mongodb://127.0.0.
|   expenses: 47000
| },
| {
|   animal_name: "Spotted Deer",
|   nature: "harmless",
|   favorite_foods: ["grass", "leaves"],
|   care_taker_name: "Suman Patel",
|   life_span: 10,
|   timestamp: new Date("2021-09-18"),
|   expenses: 15000
| },
| {
|   animal_name: "Grey Wolf",
|   nature: "harm",
|   favorite_foods: ["meat", "rabbit"],
|   care_taker_name: "Raghav Sharma",
|   life_span: 12,
|   timestamp: new Date("2018-11-05"),
|   expenses: 30000
| },
| {
|   animal_name: "Peacock",
|   nature: "harmless",
|   favorite_foods: ["grains", "insects"],
|   care_taker_name: "Meena Joshi",
|   life_span: 20,
|   timestamp: new Date("2022-01-10"),
|   expenses: 8000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69a06eb080f9d99fa67c2911'),
    '1': ObjectId('69a06eb080f9d99fa67c2912'),
    '2': ObjectId('69a06eb080f9d99fa67c2913'),
    '3': ObjectId('69a06eb080f9d99fa67c2914'),
    '4': ObjectId('69a06eb080f9d99fa67c2915')
  }
}
animal> |
```

5. Add 5 domestic-animal details to the collection named 'domestic_animals'. Each document consists of following fields as animal_name, gender (male or female), favorite_foods (meat, rabbits, deer etc) as array, animal_petname, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
mongosh mongodb://127.0.0.
    '4': ObjectId('69a06eb080f9d99fa67c2915')
  }
}
animal> db.domestic_animals.insertMany([
| {
|   animal_name: "Cow",
|   gender: "female",
|   favorite_foods: ["grass", "fodder"],
|   animal_petname: "Ganga",
|   life_span: 18,
|   timestamp: new Date("2017-04-14"),
|   expenses: 12000
| },
| {
|   animal_name: "Dog",
|   gender: "male",
|   favorite_foods: ["meat", "dog food"],
|   animal_petname: "Bruno",
|   life_span: 12,
|   timestamp: new Date("2021-07-20"),
|   expenses: 10000
| },
| {
|   animal_name: "Goat",
|   gender: "female",
|   favorite_foods: ["leaves", "grass"],
|   animal_petname: "Chutki",
|   life_span: 9,
|   timestamp: new Date("2019-10-05"),
|   expenses: 7000
| },
| {
|   animal_name: "Cat",
|   gender: "female",
|   favorite_foods: ["fish", "milk"],
|   animal_petname: "Misty",
|   life_span: 14,
|   timestamp: new Date("2020-12-11"),
|   expenses: 6000
| },
| {
```

```
mongosh mongodb://127.0.0.
|    expenses: 10000
| },
| {
|    animal_name: "Goat",
|    gender: "female",
|    favorite_foods: ["leaves", "grass"],
|    animal_petname: "Chutki",
|    life_span: 9,
|    timestamp: new Date("2019-10-05"),
|    expenses: 7000
| },
| {
|    animal_name: "Cat",
|    gender: "female",
|    favorite_foods: ["fish", "milk"],
|    animal_petname: "Misty",
|    life_span: 14,
|    timestamp: new Date("2020-12-11"),
|    expenses: 6000
| },
| {
|    animal_name: "Horse",
|    gender: "male",
|    favorite_foods: ["hay", "grains"],
|    animal_petname: "Thunder",
|    life_span: 25,
|    timestamp: new Date("2016-08-30"),
|    expenses: 20000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69a06efe80f9d99fa67c2916'),
    '1': ObjectId('69a06efe80f9d99fa67c2917'),
    '2': ObjectId('69a06efe80f9d99fa67c2918'),
    '3': ObjectId('69a06efe80f9d99fa67c2919'),
    '4': ObjectId('69a06efe80f9d99fa67c291a')
  }
}
animal> |
```

6. Write a MongoDB query to display all documents available in wild_animals and domastic_animals.

```
mongosh mongodb://127.0.0.
  }
}
animal> db.wild_animals.find()
[
  {
    _id: ObjectId('69a06eb080f9d99fa67c2911'),
    animal_name: 'White Bengal Tiger',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer', 'rabbit' ],
    care_taker_name: 'Raghav Sharma',
    life_span: 15,
    timestamp: ISODate('2019-06-12T00:00:00.000Z'),
    expenses: 50000
  },
  {
    _id: ObjectId('69a06eb080f9d99fa67c2912'),
    animal_name: 'African Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'zebra' ],
    care_taker_name: 'Aman Verma',
    life_span: 14,
    timestamp: ISODate('2020-03-22T00:00:00.000Z'),
    expenses: 47000
  },
  {
    _id: ObjectId('69a06eb080f9d99fa67c2913'),
    animal_name: 'Spotted Deer',
    nature: 'harmless',
    favorite_foods: [ 'grass', 'leaves' ],
    care_taker_name: 'Suman Patel',
    life_span: 10,
    timestamp: ISODate('2021-09-18T00:00:00.000Z'),
    expenses: 15000
  },
  {
    _id: ObjectId('69a06eb080f9d99fa67c2914'),
    animal_name: 'Grey Wolf',
    nature: 'harm',
    favorite_foods: [ 'meat', 'rabbit' ],
    care_taker_name: 'Raghav Sharma',
    life_span: 12,
```

```
mongosh mongodb://127.0.0.
    life_span: 12,
    timestamp: ISODate('2018-11-05T00:00:00.000Z'),
    expenses: 30000
  },
  {
    _id: ObjectId('69a06eb080f9d99fa67c2915'),
    animal_name: 'Peacock',
    nature: 'harmless',
    favorite_foods: [ 'grains', 'insects' ],
    care_taker_name: 'Meena Joshi',
    life_span: 20,
    timestamp: ISODate('2022-01-10T00:00:00.000Z'),
    expenses: 8000
  }
]
animal> db.domestic_animals.find()
[
  {
    _id: ObjectId('69a06efe80f9d99fa67c2916'),
    animal_name: 'Cow',
    gender: 'female',
    favorite_foods: [ 'grass', 'fodder' ],
    animal_petname: 'Ganga',
    life_span: 18,
    timestamp: ISODate('2017-04-14T00:00:00.000Z'),
    expenses: 12000
  },
  {
    _id: ObjectId('69a06efe80f9d99fa67c2917'),
    animal_name: 'Dog',
    gender: 'male',
    favorite_foods: [ 'meat', 'dog food' ],
    animal_petname: 'Bruno',
    life_span: 12,
    timestamp: ISODate('2021-07-20T00:00:00.000Z'),
    expenses: 10000
  },
  {
    _id: ObjectId('69a06efe80f9d99fa67c2918'),
    animal_name: 'Goat',
    gender: 'female',
```

```
mongosh mongodb://127.0.0.
    _id: ObjectId('69a06efe80f9d99fa67c2917'),
    animal_name: 'Dog',
    gender: 'male',
    favorite_foods: [ 'meat', 'dog food' ],
    animal_petname: 'Bruno',
    life_span: 12,
    timestamp: ISODate('2021-07-20T00:00:00.000Z'),
    expenses: 10000
  },
  {
    _id: ObjectId('69a06efe80f9d99fa67c2918'),
    animal_name: 'Goat',
    gender: 'female',
    favorite_foods: [ 'leaves', 'grass' ],
    animal_petname: 'Chutki',
    life_span: 9,
    timestamp: ISODate('2019-10-05T00:00:00.000Z'),
    expenses: 7000
  },
  {
    _id: ObjectId('69a06efe80f9d99fa67c2919'),
    animal_name: 'Cat',
    gender: 'female',
    favorite_foods: [ 'fish', 'milk' ],
    animal_petname: 'Misty',
    life_span: 14,
    timestamp: ISODate('2020-12-11T00:00:00.000Z'),
    expenses: 6000
  },
  {
    _id: ObjectId('69a06efe80f9d99fa67c291a'),
    animal_name: 'Horse',
    gender: 'male',
    favorite_foods: [ 'hay', 'grains' ],
    animal_petname: 'Thunder',
    life_span: 25,
    timestamp: ISODate('2016-08-30T00:00:00.000Z'),
    expenses: 20000
  }
]
animal>
```

7. Write a MongoDB query to display only animal name and expenses in all the collection of the database

```
mongosh mongodb://127.0.0.
    timestamp: ISODate('2019-10-05T00:00:00.000Z'),
    expenses: 7000
  },
  {
    _id: ObjectId('69a06efe80f9d99fa67c2919'),
    animal_name: 'Cat',
    gender: 'female',
    favorite_foods: [ 'fish', 'milk' ],
    animal_petname: 'Misty',
    life_span: 14,
    timestamp: ISODate('2020-12-11T00:00:00.000Z'),
    expenses: 6000
  },
  {
    _id: ObjectId('69a06efe80f9d99fa67c291a'),
    animal_name: 'Horse',
    gender: 'male',
    favorite_foods: [ 'hay', 'grains' ],
    animal_petname: 'Thunder',
    life_span: 25,
    timestamp: ISODate('2016-08-30T00:00:00.000Z'),
    expenses: 20000
  }
]
animal> db.wild_animals.find({}, { animal_name: 1, expenses: 1, _id: 0 })
[
  { animal_name: 'White Bengal Tiger', expenses: 50000 },
  { animal_name: 'African Lion', expenses: 47000 },
  { animal_name: 'Spotted Deer', expenses: 15000 },
  { animal_name: 'Grey Wolf', expenses: 30000 },
  { animal_name: 'Peacock', expenses: 8000 }
]
animal> db.domestic_animals.find({}, { animal_name: 1, expenses: 1, _id: 0 })
[
  { animal_name: 'Cow', expenses: 12000 },
  { animal_name: 'Dog', expenses: 10000 },
  { animal_name: 'Goat', expenses: 7000 },
  { animal_name: 'Cat', expenses: 6000 },
  { animal_name: 'Horse', expenses: 20000 }
]
animal>
```

8. Write a MongoDB query to display domestic_animals whose life is a particular year

```
mongosh mongodb://127.0.0.
  {
    _id: ObjectId('69a06efe80f9d99fa67c291a'),
    animal_name: 'Horse',
    gender: 'male',
    favorite_foods: [ 'hay', 'grains' ],
    animal_petname: 'Thunder',
    life_span: 25,
    timestamp: ISODate('2016-08-30T00:00:00.000Z'),
    expenses: 20000
  }
]
animal> db.wild_animals.find({}, { animal_name: 1, expenses: 1, _id: 0 })
[
  { animal_name: 'White Bengal Tiger', expenses: 50000 },
  { animal_name: 'African Lion', expenses: 47000 },
  { animal_name: 'Spotted Deer', expenses: 15000 },
  { animal_name: 'Grey Wolf', expenses: 30000 },
  { animal_name: 'Peacock', expenses: 8000 }
]
animal> db.domestic_animals.find({}, { animal_name: 1, expenses: 1, _id: 0 })
[
  { animal_name: 'Cow', expenses: 12000 },
  { animal_name: 'Dog', expenses: 10000 },
  { animal_name: 'Goat', expenses: 7000 },
  { animal_name: 'Cat', expenses: 6000 },
  { animal_name: 'Horse', expenses: 20000 }
]
animal> db.domestic_animals.find({ life_span: 12 })
[
  {
    _id: ObjectId('69a06efe80f9d99fa67c2917'),
    animal_name: 'Dog',
    gender: 'male',
    favorite_foods: [ 'meat', 'dog food' ],
    animal_petname: 'Bruno',
    life_span: 12,
    timestamp: ISODate('2021-07-20T00:00:00.000Z'),
    expenses: 10000
  }
]
animal>
```

9. Write a MongoDB query to display wild_animals available under a particular care_taker

```
mongosh mongodb://127.0.0.
  { animal_name: 'Goat', expenses: 7000 },
  { animal_name: 'Cat', expenses: 6000 },
  { animal_name: 'Horse', expenses: 20000 }
]
animal> db.domestic_animals.find({ life_span: 12 })
[
  {
    _id: ObjectId('69a06efe80f9d99fa67c2917'),
    animal_name: 'Dog',
    gender: 'male',
    favorite_foods: [ 'meat', 'dog food' ],
    animal_petname: 'Bruno',
    life_span: 12,
    timestamp: ISODate('2021-07-20T00:00:00.000Z'),
    expenses: 10000
  }
]
animal> db.wild_animals.find({ care_taker_name: "Raghav Sharma" })
[
  {
    _id: ObjectId('69a06eb080f9d99fa67c2911'),
    animal_name: 'White Bengal Tiger',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer', 'rabbit' ],
    care_taker_name: 'Raghav Sharma',
    life_span: 15,
    timestamp: ISODate('2019-06-12T00:00:00.000Z'),
    expenses: 50000
  },
  {
    _id: ObjectId('69a06eb080f9d99fa67c2914'),
    animal_name: 'Grey Wolf',
    nature: 'harm',
    favorite_foods: [ 'meat', 'rabbit' ],
    care_taker_name: 'Raghav Sharma',
    life_span: 12,
    timestamp: ISODate('2018-11-05T00:00:00.000Z'),
    expenses: 30000
  }
]
animal>
```

10. Write a MongoDB query to display animal name, favorite_foods and expenses details whose lifespan is more than 5 years.

```
mongosh mongodb://127.0.0.
    expenses: 30000
  }
]
animal> db.wild_animals.find(
|   { life_span: { $gt: 5 } },
|   { animal_name: 1, favorite_foods: 1, expenses: 1, _id: 0 }
| )
[
  {
    animal_name: 'White Bengal Tiger',
    favorite_foods: [ 'meat', 'deer', 'rabbit' ],
    expenses: 50000
  },
  {
    animal_name: 'African Lion',
    favorite_foods: [ 'meat', 'zebra' ],
    expenses: 47000
  },
  {
    animal_name: 'Spotted Deer',
    favorite_foods: [ 'grass', 'leaves' ],
    expenses: 15000
  },
  {
    animal_name: 'Grey Wolf',
    favorite_foods: [ 'meat', 'rabbit' ],
    expenses: 30000
  },
  {
    animal_name: 'Peacock',
    favorite_foods: [ 'grains', 'insects' ],
    expenses: 8000
  }
]
animal> db.domestic_animals.find(
|   { life_span: { $gt: 5 } },
|   { animal_name: 1, favorite_foods: 1, expenses: 1, _id: 0 }
| )
[
  {
    animal_name: 'Cow',
```

```
mongosh mongodb://127.0.0.
    favorite_foods: [ 'meat', 'rabbit' ],
    expenses: 30000
  },
  {
    animal_name: 'Peacock',
    favorite_foods: [ 'grains', 'insects' ],
    expenses: 8000
  }
]
animal> db.domestic_animals.find(
|   { life_span: { $gt: 5 } },
|   { animal_name: 1, favorite_foods: 1, expenses: 1, _id: 0 }
| )
[
  {
    animal_name: 'Cow',
    favorite_foods: [ 'grass', 'fodder' ],
    expenses: 12000
  },
  {
    animal_name: 'Dog',
    favorite_foods: [ 'meat', 'dog food' ],
    expenses: 10000
  },
  {
    animal_name: 'Goat',
    favorite_foods: [ 'leaves', 'grass' ],
    expenses: 7000
  },
  {
    animal_name: 'Cat',
    favorite_foods: [ 'fish', 'milk' ],
    expenses: 6000
  },
  {
    animal_name: 'Horse',
    favorite_foods: [ 'hay', 'grains' ],
    expenses: 20000
  }
]
animal> |
```

27°C
Mostly clear
Search
ENG
IN
21:38
26-02-2026